Practical - 10

Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

A. Theory

1. What is a PHP session?

Answer: A session stores server-side information associated with a visitor, allowing the application to remember that a user has authenticated across multiple page requests.

2. Why regenerate the session ID after login?

Answer: session_regenerate_id(true) helps prevent session fixation by replacing the session identifier after authentication.

3. What is role-based access control (RBAC)?

Answer: RBAC grants access according to a user's assigned role, such as admin or student. The role must be checked on the server for every protected action.

4. How is a session timeout implemented?

Answer: Store a last-activity timestamp in the session. If the inactivity period exceeds the limit, clear the session and require the user to log in again.

5. What is a remember-me token?

Answer: It is an optional long-lived random token stored in a secure cookie, with only a hash of the token stored in the database. It should be random, revocable, and protected with Secure/HttpOnly/SameSite cookie settings.

B. Implementation / Procedure

1. Login form

Create an email/username and password form that submits using POST.

2. Verify credentials

Fetch the matching user using a prepared statement, then validate the password with password_verify(). Use a generic error for invalid credentials.

3. Start authenticated session

After success, regenerate the session ID and store only necessary identifiers such as user ID, name, and role in the session.

4. Protect pages

At the top of each protected page, verify login state and role before rendering protected content.

5. Timeout and logout

Check the last-activity timestamp on each protected request. On timeout or logout, clear session variables, invalidate the session cookie, and call session_destroy().

6. Role redirection

Redirect administrators and students to their respective dashboards. Do not rely only on hiding menu links; enforce permissions on the server.

C. Test Cases

Test Case	Input / Action	Expected Result
Correct credentials	Valid registered user and password	Correct dashboard opens
Wrong password	Correct account with incorrect password	Generic login error appears
Logout	Click logout	Protected page cannot be opened without logging in again
Timeout	Wait beyond configured inactivity period	Session expires and login is required
Role restriction	Student opens admin URL directly	Access is denied or redirected

D. Key Questions / Analysis / Interpretation

1. Are sessions started and destroyed securely?

Answer: The session is started before protected page output. On logout, session data is cleared, the session cookie is invalidated where appropriate, and the session is destroyed.

2. Is password_verify() used correctly?

Answer: The submitted password is checked against the stored hash using password_verify(); the plain password is not compared directly with the hash.

3. Is unauthorized dashboard access blocked?

Answer: Yes. Every protected page checks whether the user is logged in and whether the user's role is permitted. A user who fails the check is redirected or shown an access-denied message.

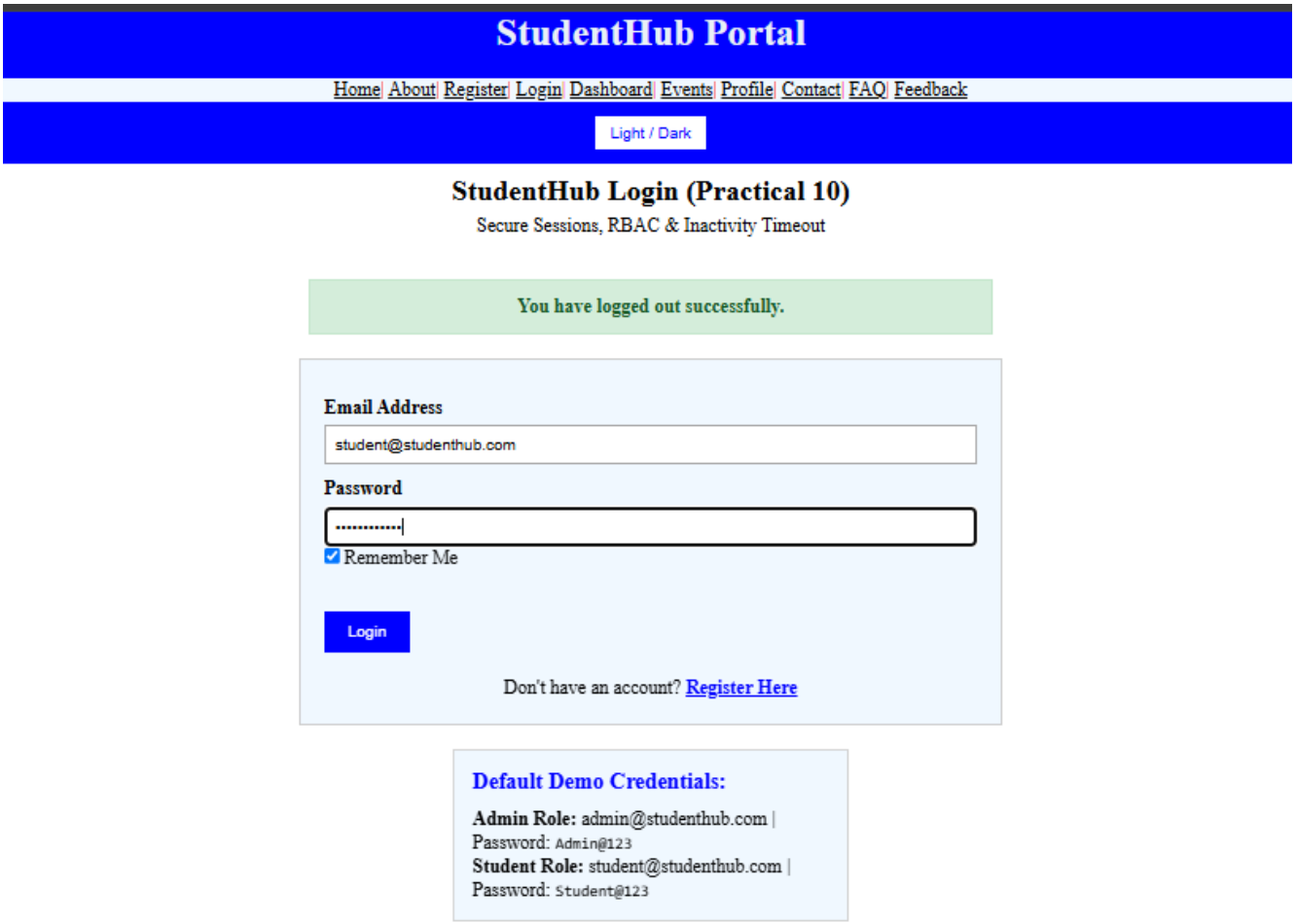
4. Is session timeout implemented?

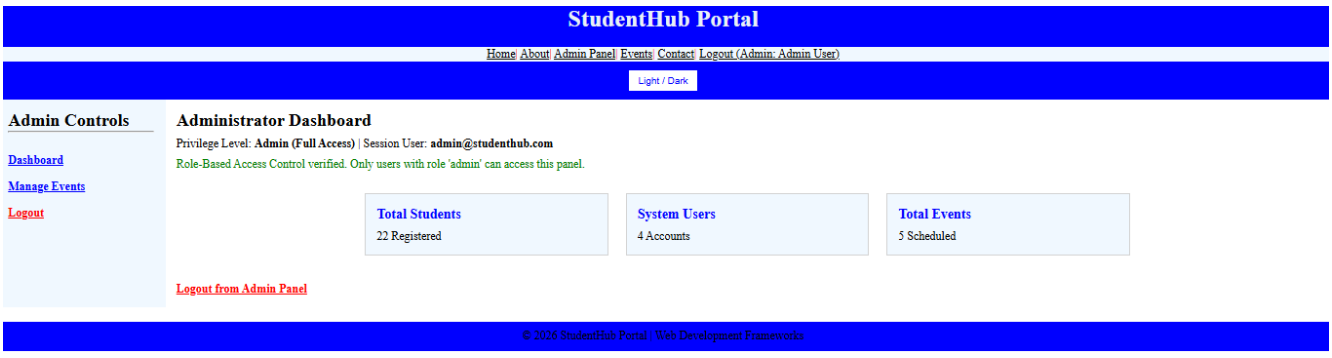
Answer: The application stores the last activity time and ends the session when the configured inactivity limit is exceeded.

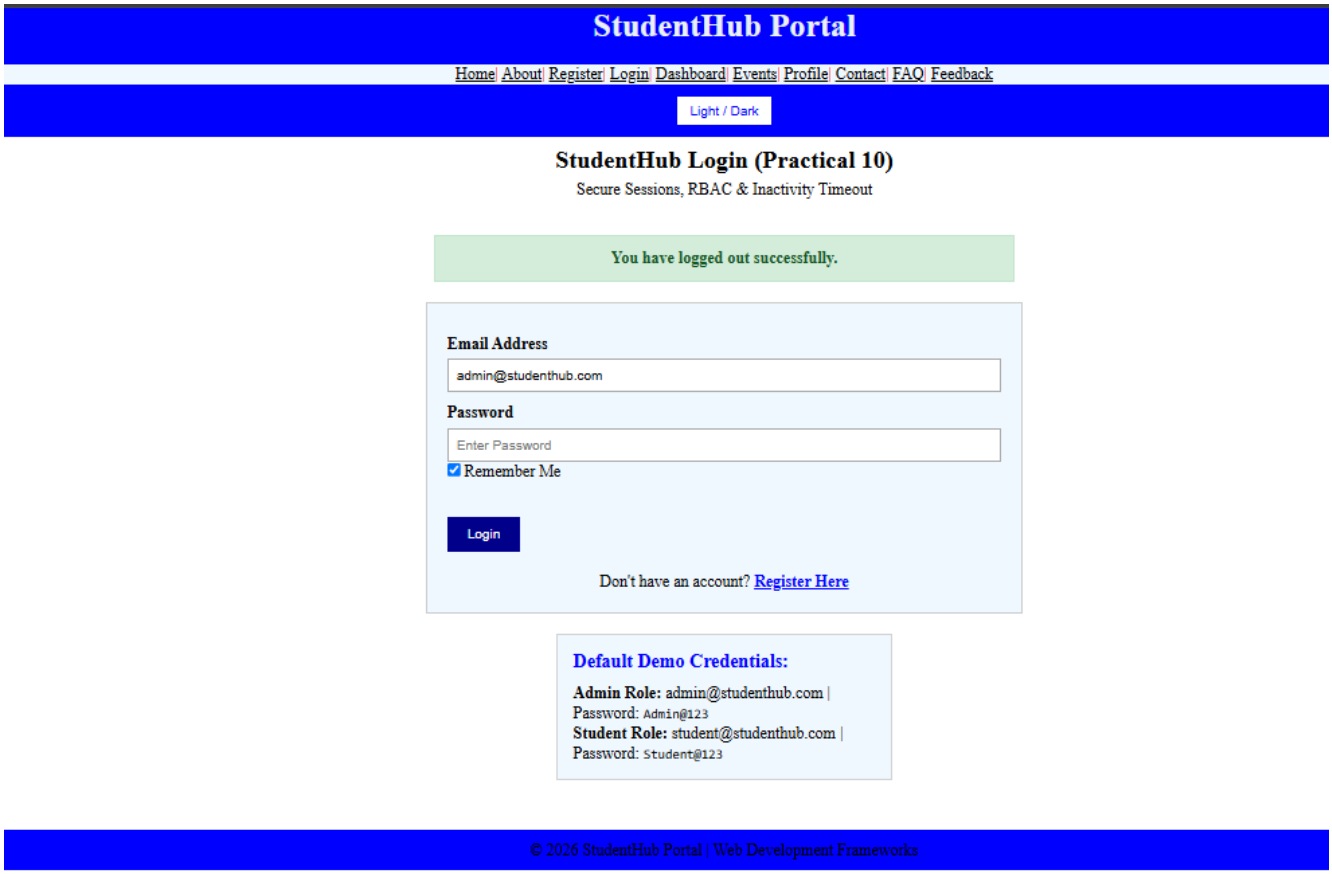
5. Are admin and student users redirected appropriately?

Answer: After successful authentication, the role value determines the destination dashboard, such as the admin dashboard for administrators and student dashboard for students.

M. Evidence of Work (Screenshots)







N.

N. GitHub Repository Link

https://github.com/vatsal-1950/StudentHUB.git